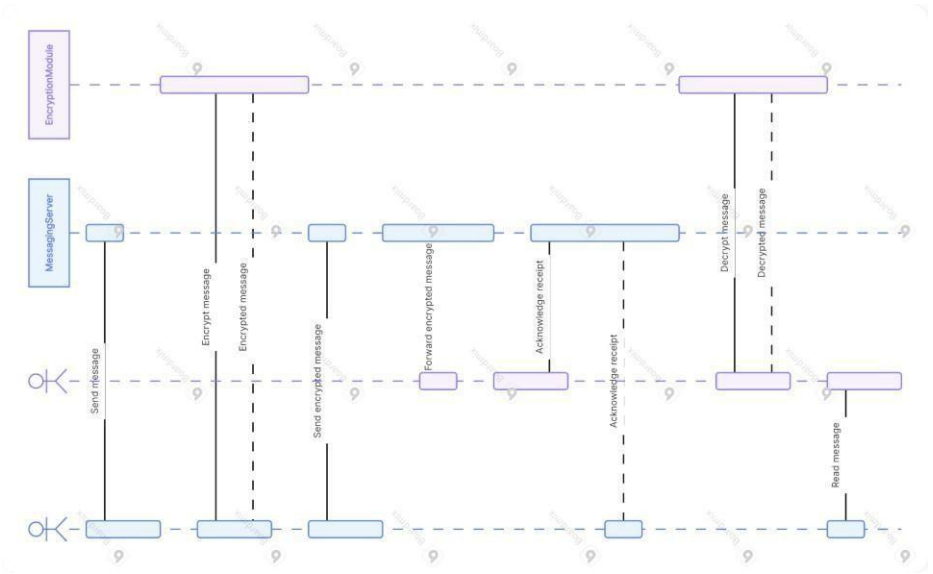
DAYANANDA SAGAR COLLEGE OF ENGINEERING

(An Autonomous Institute affiliated to Visvesvaraya Technological University (VTU), Belagavi,
Approved by AICTE and UGC, Accredited by NAAC with 'A' grade & ISO 9001-2015 Certified Institution)

Shavige Malleshwara Hills, Kumaraswamy Layout, Bengaluru - 560 111. India

DEPARTMENT	**ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING**				
Semester: VII	**Course Title: Data Security & Privacy Lab**				**Course Code: 22AI73**
PROGRAM QUESTION	6a) Develop a real-time messaging application where two devices can communicate securely. The application must use Transport Layer Security (TLS) for securing the client-server communication and End-to-End Encryption (E2EE) to ensure that only the sender and recipient can access the message content. The server should act solely as a message relay and have no access to plaintext messages. 6b) Draw a sequence diagram and Build a real-world messaging application where two devices can communicate securely using end-to-end encryption (E2EE). The application should ensure that only the sender and receiver can read the messages, and even the server facilitating the communication cannot access the plaintext messages.				
STUDENT NAMES	Udayaraj		USN	1DS22AI057	
LAB MARKS	**Program Execution (10)**	**Source Code (10)**	**Result (5)**	**Innovation/ Extra Features (5)**	**Total**
TOOLS USED	Cursor				
MAP COURSE OUTCOME	CO1				
MAP PROGRAM OUTCOME	PO1, PO2, PO3, PO5, PO6, PO7, PO9, PO10, PO11				
MAP PROGRAM SPECIFIC OUTCOME	PSO1, PSO2, PSO3				

UML / SEQUENCE DIAGRAM	
COURSE COORDINATOR	Dr ARUNA M G & Prof ENSTEIH SILVIA

Code for Program 6:

6 a) Develop a real-time messaging application where two devices can communicate securely. The application must use Transport Layer Security (TLS) for securing the client-server communication and End-to-End Encryption (E2EE) to ensure that only the sender and recipient can access the message content. The server should act solely as a message relay and have no access to plaintext messages.

Server.py

```
import socket, ssl, threading, base64
from nacl.public import PrivateKey, PublicKey, Box

HOST = '127.0.0.1'
PORT = 8888
```

```
# Generate server keypair
server_private = PrivateKey.generate()
server_public = server_private.public_key
```

```
clients = []  # list of (conn, box)
```

```
def handle_client(conn, addr, box):
    print(f"[+] Client connected: {addr}")
    try:
        while True:
            data = conn.recv(4096)
            if not data:
                break
            print(f"[Encrypted from {addr}] {base64.b64encode(data).decode()}")
            try:
                plaintext = box.decrypt(data).decode()
                print(f"[Decrypted] {plaintext}")
            except Exception as e:
                print(f"[Decrypt Error] {e}")
                continue
```

```
            # broadcast plaintext (re-encrypt with each client's box)
            for c, b in clients[:]:
                if c != conn:
                    try:
                        c.send(b.encrypt(plaintext.encode()))
                    except Exception as e:
```

```
                        print(f"[Send Error] {e}, removing client")
                        clients.remove((c, b))
    finally:
        print(f"[-] Client disconnected: {addr}")
        clients.remove((conn, box))
        conn.close()
```

```
def client_thread(ssl_conn, addr):
    # Step 1: exchange public keys
    ssl_conn.send(server_public.encode())  # send server public key
    client_pub = PublicKey(ssl_conn.recv(32))  # receive client public key
    box = Box(server_private, client_pub)
    clients.append((ssl_conn, box))
    handle_client(ssl_conn, addr, box)
```

```
def main():
    context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
    context.load_cert_chain(certfile="cert.pem", keyfile="key.pem")
```

```
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
        sock.bind((HOST, PORT))
        sock.listen(5)
        print(f"[*] Server listening on {HOST}:{PORT}")
        with context.wrap_socket(sock, server_side=True) as ssock:
            while True:
                conn, addr = ssock.accept()
                threading.Thread(target=client_thread, args=(conn, addr), daemon=True).start()
```

```
if __name__ == "__main__":
    main()
```

client.py

```
import socket, ssl, threading, tkinter as tk
from nacl.public import PrivateKey, PublicKey, Box

HOST = '127.0.0.1'
PORT = 8888
```

```
class ChatClient:
    def __init__(self, root):
        self.root = root
        self.root.title("Secure Chat")
```

```
        # Two chat areas
        self.text_local = tk.Text(root, height=15, width=40, bg="lightyellow")
        self.text_remote = tk.Text(root, height=15, width=40, bg="lightblue")
        self.text_local.grid(row=0, column=0, padx=5, pady=5)
        self.text_remote.grid(row=0, column=1, padx=5, pady=5)
```

```
        self.entry = tk.Entry(root, width=60)
        self.entry.grid(row=1, column=0, columnspan=2, padx=5, pady=5)
        self.send_btn = tk.Button(root, text="Send", command=self.send_msg)
        self.send_btn.grid(row=1, column=2, padx=5)
```

```
        self.sock = None
        self.box = None
        self.connect()
```

```
    def connect(self):
        context = ssl.create_default_context()
        context.check_hostname = False
        context.verify_mode = ssl.CERT_NONE
```

```
        raw_sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.sock = context.wrap_socket(raw_sock, server_hostname=HOST)
        self.sock.connect((HOST, PORT))
```

```
        # key exchange
        client_private = PrivateKey.generate()
        client_public = client_private.public_key
        server_pub = PublicKey(self.sock.recv(32))
        self.sock.send(bytes(client_public))
        self.box = Box(client_private, server_pub)
```

```
        threading.Thread(target=self.receive_msgs, daemon=True).start()
```

```
    def send_msg(self):
        msg = self.entry.get()
        if msg:
            self.text_local.insert(tk.END, f"You: {msg}\n")
            self.entry.delete(0, tk.END)
            ciphertext = self.box.encrypt(msg.encode())
            self.sock.send(ciphertext)
```

```
    def receive_msgs(self):
        while True:
            try:
                data = self.sock.recv(4096)
                if not data:
                    break
                msg = self.box.decrypt(data).decode()
                self.text_remote.insert(tk.END, f"Peer: {msg}\n")
            except Exception:
                break
```

```
if __name__ == "__main__":
    root = tk.Tk()  # main window
```

```
# first chat client
client1 = ChatClient(root)
```

```
# second chat window
second_window = tk.Toplevel(root)  # new popup
client2 = ChatClient(second_window)
```

```
root.mainloop()
```

Generate.py

```
from cryptography import x509
from cryptography.x509.oid import NameOID
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.backends import default_backend
from cryptography.hazmat.primitives import serialization
from cryptography.hazmat.primitives.asymmetric import rsa
import datetime

def generate_self_signed_cert():
    """
    Generates a self-signed certificate and a private key.
    """
    # Generate our key
    key = rsa.generate_private_key(
        public_exponent=65537,
        key_size=2048,
        backend=default_backend()
    )
```

```
    # Write our key to disk for safe keeping
    with open("key.pem", "wb") as f:
        f.write(key.private_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PrivateFormat.TraditionalOpenSSL,
            encryption_algorithm=serialization.NoEncryption(),
```

```
        ))
```

```
    # Various details about who we are. For a self-signed certificate the
    # subject and issuer are always the same.
    subject = issuer = x509.Name([
        x509.NameAttribute(NameOID.COUNTRY_NAME, u"US"),
        x509.NameAttribute(NameOID.STATE_OR_PROVINCE_NAME, u"California"),
        x509.NameAttribute(NameOID.LOCALITY_NAME, u"San Francisco"),
        x509.NameAttribute(NameOID.ORGANIZATION_NAME, u"My Company"),
        x509.NameAttribute(NameOID.COMMON_NAME, u"localhost"),
    ])
```

```
    cert = x509.CertificateBuilder().subject_name(
        subject
    ).issuer_name(
        issuer
    ).public_key(
        key.public_key()
    ).serial_number(
        x509.random_serial_number()
    ).not_valid_before(
        datetime.datetime.utcnow()
    ).not_valid_after(
        # Our certificate will be valid for 10 days
        datetime.datetime.utcnow() + datetime.timedelta(days=10)
    ).add_extension(
        x509.SubjectAlternativeName([x509.DNSName(u"localhost")]),
        critical=False,
    # Sign our certificate with our private key
    ).sign(key, hashes.SHA256(), default_backend())
```

```
    # Write our certificate out to disk.
    with open("cert.pem", "wb") as f:
        f.write(cert.public_bytes(serialization.Encoding.PEM))
```

```
if __name__ == "__main__":
    generate_self_signed_cert()
    print("Certificate and private key generated: cert.pem, key.pem")
```

```
from cryptography import x509
from cryptography.x509.oid import NameOID
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.backends import default_backend
from cryptography.hazmat.primitives import serialization
from cryptography.hazmat.primitives.asymmetric import rsa
import datetime

def generate_self_signed_cert():
    """
    Generates a self-signed certificate and a private key.
    """
    # Generate our key
    key = rsa.generate_private_key(
        public_exponent=65537,
        key_size=2048,
        backend=default_backend()
    )
```

```
    # Write our key to disk for safe keeping
    with open("key.pem", "wb") as f:
        f.write(key.private_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PrivateFormat.TraditionalOpenSSL,
            encryption_algorithm=serialization.NoEncryption(),
        ))
```

```
    # Various details about who we are. For a self-signed certificate the
    # subject and issuer are always the same.
    subject = issuer = x509.Name([
        x509.NameAttribute(NameOID.COUNTRY_NAME, u"US"),
        x509.NameAttribute(NameOID.STATE_OR_PROVINCE_NAME, u"California"),
```

```
        x509.NameAttribute(NameOID.LOCALITY_NAME, u"San Francisco"),
        x509.NameAttribute(NameOID.ORGANIZATION_NAME, u"My Company"),
        x509.NameAttribute(NameOID.COMMON_NAME, u"localhost"),
    ])
```

```
    cert = x509.CertificateBuilder().subject_name(
        subject
    ).issuer_name(
        issuer
    ).public_key(
        key.public_key()
    ).serial_number(
        x509.random_serial_number()
    ).not_valid_before(
        datetime.datetime.utcnow()
    ).not_valid_after(
        # Our certificate will be valid for 10 days
        datetime.datetime.utcnow() + datetime.timedelta(days=10)
    ).add_extension(
        x509.SubjectAlternativeName([x509.DNSName(u"localhost")]),
        critical=False,
    # Sign our certificate with our private key
    ).sign(key, hashes.SHA256(), default_backend())
```

```
    # Write our certificate out to disk.
    with open("cert.pem", "wb") as f:
        f.write(cert.public_bytes(serialization.Encoding.PEM))
```

```
if __name__ == "__main__":
    generate_self_signed_cert()
    print("Certificate and private key generated: cert.pem, key.pem")
```

Crypto.py

```
from cryptography.fernet import Fernet

def generate_key():
    """Generates a key for encryption."""
    return Fernet.generate_key()
```

```
def encrypt_message(key, message):
    """Encrypts a message using the provided key."""
    f = Fernet(key)
    encrypted_message = f.encrypt(message.encode())
    return encrypted_message
```

```
def decrypt_message(key, encrypted_message):
    """Decrypts a message using the provided key."""
    f = Fernet(key)
    decrypted_message = f.decrypt(encrypted_message).decode()
    return decrypted_message
```

RESULT:


Secure Chat
You: hii
Peer: hlo
Send
Stop
Deploy
Secure Chat
You: hlo
Peer: hii
Send

6 b) Draw a sequence diagram and Build a real-world messaging application where two devices can communicate securely using end-to-end encryption (E2EE). The application should ensure that only the sender and receiver can read the messages, and even the server facilitating the communication cannot access the plaintext messages.

```
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes
import base64

# Helper functions for AES encryption/decryption

def pad(s):
    return s + (16 - len(s) % 16) * chr(16 - len(s) % 16)


def unpad(s):
    return s[:-ord(s[len(s) - 1:])]


def encrypt_message(key, message):
    cipher = AES.new(key, AES.MODE_CBC)
    ct_bytes = cipher.encrypt(pad(message).encode())
    iv = base64.b64encode(cipher.iv).decode('utf-8')
    ct = base64.b64encode(ct_bytes).decode('utf-8')
    return iv + ":" + ct


def decrypt_message(key, encrypted_message):
    iv, ct = encrypted_message.split(":")
    iv = base64.b64decode(iv)
    ct = base64.b64decode(ct)
    cipher = AES.new(key, AES.MODE_CBC, iv=iv)
    pt = unpad(cipher.decrypt(ct).decode())
    return pt

# Shared secret key (for demo purposes)
key = get_random_bytes(16)

# Simulated server (just forwards encrypted messages)

def server_forward(encrypted_msg):
    print("\n[Server] Forwarding encrypted message...")
    return encrypted_msg

# Simple chat loop
print("=== Secure E2EE Chat Simulation ===")
print("Type 'exit' to quit.\n")
sender = "User A"
receiver = "User B"

while True:
    msg = input(f"{sender}: ")
    if msg.lower() == 'exit':
        print("Chat ended.")
        break

    # Encrypt before sending
    encrypted_msg = encrypt_message(key, msg)
    print(f"[{sender}] Encrypted message:", encrypted_msg)

    # Server forwards message
    forwarded_msg = server_forward(encrypted_msg)

    # Receiver decrypts
    decrypted_msg = decrypt_message(key, forwarded_msg)
    print(f"{receiver} received: {decrypted_msg}\n")

    # Swap sender and receiver for next turn
    sender, receiver = receiver, sender
```

RESULT:

```
=== Secure E2EE Chat Simulation ===
Type 'exit' to quit.

User A: hii
[User A] Encrypted message: X4l8hp6frtnewT2H+vw93g==:MhapJSL1bV0o7CL/v8oFlg==

[Server] Forwarding encrypted message...
User B received: hii

User B: hlo
[User B] Encrypted message: KQ7lcbOK64Vk3X1mHMGsKA==:dEy9FxEFFvXzk4J7yBEO+g==

[Server] Forwarding encrypted message...
User A received: hlo

User A:
```